CLAHE Implementation and Optimization using CUDA - Lab 4

High Performance Computing Systems (ECE415)

Angelos Gkertsos (03658)
Nikolaos Kalamaris (03768)

1 Introduction

The objective of this exercise was to implement and optimize the Contrast Limited Adaptive Histogram Equalization (CLAHE) algorithm on the NVIDIA CUDA architecture. CLAHE is a computer vision technique used to improve contrast in images. Unlike standard histogram equalization, which operates globally, CLAHE operates on small regions (tiles), computing local histograms and redistributing brightness values.

Our implementation focuses on parallelizing the three distinct stages of the algorithm:

1. Tiled Histogram Generation and Clipping.
2. Cumulative Distribution Function (CDF) and LUT calculation.
3. Image Rendering using Bilinear Interpolation.

This report analyzes the parallelization strategy, the optimizations applied to maximize throughput (Mpixels/sec), and the scalability of the solution on large datasets.

1.1 Device Specifications

The experiments were conducted on the NVIDIA Tesla K80 GPU by isolating the assigned GPU (Device 1). The detailed specifications of the device used for benchmarking (Device 1) are presented in Table. 1.

Table 1: NVIDIA Tesla K80 Device Specifications

Specification	**Value**
Device Name	Tesla K80
CUDA Driver / Runtime Version	11.4 / 11.5
Compute Capability	3.7 (Kepler Architecture)
Total Global Memory	11441 MBytes ($\approx$ 11.17 GB)
Multiprocessors (MPs)	13
CUDA Cores	2496 (192 per MP)
GPU Max Clock Rate	824 MHz (0.82 GHz)
Memory Clock Rate	2505 MHz
Memory Bus Width	384-bit
L2 Cache Size	1.5 MB (1572864 bytes)
Memory Limits	
Total Constant Memory	65536 bytes (64 KB)
Total Shared Memory per Block	49152 bytes (48 KB)
Total Shared Memory per MP	114688 bytes (112 KB)
Total Registers per Block	65536
Thread Configuration	
Warp Size	32
Max Threads per Multiprocessor	2048
Max Threads per Block	1024
Max Block Dimensions (x, y, z)	$(1024, 1024, 64)$
Max Grid Dimensions (x, y, z)	$(2147483647, 65535, 65535)$
Features	
Concurrent Copy and Kernel Exec.	Yes (2 copy engines)
ECC Support	Enabled
Unified Addressing (UVA)	Yes
Managed Memory	Yes
PCI Domain / Bus / Location ID	0 / 7 / 0

- **CUDA Capability:** 3.7 (Kepler Architecture).
- **Total Global Memory:** 11,441 MB ($\approx$ 11.99 GB).
- **Maximum Threads per Block:** 1024.

2 Implementation Strategy - Optimizations

To achieve maximum performance on the Tesla K80 GPU, we implemented a highly optimized pipeline that combines efficient memory management, latency hiding techniques, and hardware-specific caching mechanisms. Our strategy diverges from a naive port by introducing asynchronous processing and texture caching.

2.1 Kernel Architecture

The workload was decomposed into two specialized kernels to separate the histogram generation phase from the rendering phase:

- **Kernel 1 (Histogram & LUT Generation):** This kernel operates on a per-tile basis.
  - **Shared Memory Histograms:** To minimize collisions in Global Memory, each thread block computes a local histogram in Shared Memory using `atomicAdd`. Only the final aggregated result (the LUT) is written to Global Memory.
  - **Parallel Prefix Sum (Scan):** Instead of a serial CPU calculation, we implemented a parallel Hillis-Steele scan within the kernel to convert the histogram into a Cumulative Distribution Function (CDF) and subsequently generate the Look-Up Table (LUT) in parallel.
    $$x[i] = x[i] + x[i - 2^k] \tag{1}$$
    Threads cooperate in $\log_2(256)$ steps to compute the cumulative sum in parallel, utilizing `__syncthreads()` barriers between steps.
  - **Bandwidth Optimization:** We utilized `unsigned char` (1 byte) instead of `int` (4 bytes) for storing the LUTs. Since pixel values range from 0-255, this reduces the memory bandwidth requirement for writing and reading LUTs by 75%.
- **Kernel 2 (Rendering & Interpolation):** This kernel calculates the final pixel values.
  - **Texture Memory Caching:** During the rendering phase, each thread must fetch values from four different LUTs corresponding to neighboring tiles to perform bilinear interpolation. Accessing these values directly from Global Memory results in high latency due to the non-sequential access pattern. To solve this, we bound the LUT array to a **Texture Reference** (`texLUT`) and accessed data using the `tex1Dfetch` operation. This directs memory requests through the GPU's dedicated Texture Cache. Since neighboring pixels typically access neighboring LUT values, the texture cache exploits this spatial locality, serving data almost instantly and drastically reducing the bandwidth pressure on Global Memory.

2.2 Grid and Block Configuration

The input image is divided into tiles of size 32×32 pixels (TILESIZE x TILESIZE).

- **Histogram Kernel Strategy (256 vs 1024 threads):** For the 32×32 pixel tiles, a one-to-one mapping (1024 threads) might seem intuitive. However, we explicitly selected a block size of **256 threads**, where each thread processes 4 pixels sequentially. This design choice yields three critical performance benefits:
  1. **Higher Occupancy & Latency Hiding:** Using fewer threads per block reduces **Register Pressure**. If we utilized 1024 threads, the register usage could limit the number of active blocks per Streaming Multiprocessor (SM). By using 256 threads, the GPU scheduler can fit more blocks simultaneously on the SM, effectively hiding memory latency by switching between warps.
  2. **Reduced Atomic Contention:** Since all threads update the same local histogram in Shared Memory using `atomicAdd`, having 1024 concurrent threads would lead to high serialization (conflicts) on specific bins. Reducing the concurrency to 256 threads alleviates this bottleneck.

3. **Instruction Efficiency:** The loop overhead for processing 4 pixels per thread is negligible compared to the gains from improved occupancy and reduced memory contention.

- **Render Kernel Strategy (1024 threads):** In contrast to the histogram generation, the rendering phase is a parallel operation with no inter-thread communication or synchronization requirements (no race conditions). Consequently, we maximized the block size to the hardware limit of **1024 threads** (32×32). This massive parallelism is crucial for **Latency Hiding**. Since the kernel relies heavily on fetching data from Texture Memory (which can cause latency upon cache misses), the GPU scheduler can instantly switch to other threads while waiting for memory requests to be fulfilled, ensuring the execution units remain fully utilized.

2.3 Concurrency and Latency Hiding (Streams)

To maximize the utilization of the Tesla K80's resources and hide the significant latency of data transfers over the PCIe bus, we implemented an asynchronous streaming pipeline. This approach allows the GPU to perform computations (Kernel Execution) and data transfers (Memcpy) simultaneously.

- **Pinned (Page-Locked) Memory:** Standard `malloc` allocates pageable host memory, which the GPU cannot access directly. This forces the driver to perform an extra CPU copy to a temporary buffer before transferring data. To eliminate this overhead, we allocated the host buffers using `cudaMallocHost`. This locks the memory pages in physical RAM, enabling the GPU's Direct Memory Access engine to read/write data at maximum bandwidth without CPU intervention.
- **4-Way Concurrency Strategy:** We decomposed the image domain into 4 horizontal strips (chunks). Each chunk is assigned to a unique `cudaStream`. This creates four independent execution queues, allowing the GPU scheduler to overlap operations: for example, while Stream 0 is executing the Histogram kernel, the Copy Engine can simultaneously transfer data for Stream 1.
- **Execution Flow and Synchronization:** Unlike standard streaming algorithms, CLAHE introduces a data dependency between tiles (rendering a pixel requires LUTs from neighboring tiles). This necessitated a split-phase pipeline with a global barrier:

  1. **Phase 1 (Map):** In a loop, we asynchronously dispatch the Host-to-Device transfer and the *Histogram Generation Kernel* for each stream.
  2. **Global Barrier:** This barrier ensures the entire grid's LUTs are populated before rendering begins. We invoke `cudaDeviceSynchronize()`. This step is critical because bilinear interpolation for a specific tile requires valid LUTs from its neighbors (Right, Bottom, Bottom-Right), which may belong to a different stream. If we attempted to render a tile before its neighbors were processed, we would read invalid data.
  3. **Phase 2 (Reduce):** Once safe, we launch the Render Kernel and the Device-to-Host transfer for each stream, again overlapping execution with data movement.

2.4 Shared Memory Optimization and Bank Conflict Mitigation

The assignment guidelines explicitly highlight the risk of Shared Memory Bank Conflicts. In the NVIDIA Kepler architecture (Tesla K80), shared memory is divided into 32 banks. A conflict

occurs when multiple threads in a warp access different memory addresses that map to the same bank, causing the accesses to be serialized. To address this, we implemented a **Memory Padding Strategy**.

- **Conflict-Free Indexing (Padding):** Standard parallel reduction and scan algorithms often produce strided access patterns that lead to n-way bank conflicts. To mitigate this, we implemented a macro-based padding scheme:

```
#define CONFLICT_FREE_OFFSET(n) ((n) >> LOG_NUM_BANKS)
#define SHM_IDX(n) ((n) + CONFLICT_FREE_OFFSET(n))
```

  By inserting a dummy word every 32 integers (one memory bank width), we shift the mapping of logical indices to physical memory banks. This ensures that during the Parallel Scan (CDF generation), threads with strided access patterns access distinct banks simultaneously, significantly increasing memory throughput.

- **Histogram Accumulation:** While the histogram generation involves data-dependent atomic operations (which inherently cause serialization when threads update the same bin), the use of the `SHM_IDX` macro ensures that the base structure of the shared memory array does not introduce *structural* bank conflicts unrelated to data collisions.

2.5 Warp Divergence Mitigation: Branchless Implementation

In SIMT (Single Instruction, Multiple Threads) architectures, performance degrades when threads within a single warp diverge (i.e., follow different execution paths due to conditional statements). To maximize occupancy and instruction throughput, we refactored the kernel logic to be **Branchless**.

- **Predicated Execution (Histogram):** Instead of using an `if` statement to check if a pixel is within image bounds before adding it to the histogram, we utilized predicated arithmetic:

```
int is_active = (global_x < c_width) && (global_y < c_height);
atomicAdd(&s_hist[SHM_IDX(val)], is_active);
```

  Here, `is_active` evaluates to 1 for valid pixels and 0 for padding. Since `atomicAdd` with 0 is a safe no-op, all threads in the warp execute the exact same instructions, eliminating divergence penalty.

- **Hardware Intrinsics for Clamping:** For both the histogram clipping and the bilinear interpolation boundary handling, we replaced branching logic (e.g., `if (val > limit) val = limit;`) with hardware-accelerated `min()` and `max()` instructions:

$$clipped\_val = \min(bin\_val, c\_clip\_limit) \tag{2}$$

$$x_{clamped} = \max(0, \min(grid\_w - 1, x_{raw})) \tag{3}$$

  These map directly to single assembly instructions (e.g., `IMIN`, `IMAX`), ensuring a strictly linear instruction pipeline without jump instructions or serialization masks.

2.6 Algorithmic Analysis - Histogram Redistribution Accuracy

Beyond structural parallelism and optimizations, we analyzed the numerical fidelity of the algorithm compared to the reference implementation.

We analyzed the histogram clipping and redistribution logic provided in the reference code:

```
int avg_inc = total_excess / 256;
s_hist[tx] += avg_inc;
```

We observed that using integer division for `avg_inc` causes the truncation of the remainder ($total\_excess \pmod{256}$). This implies that up to 255 pixel counts are "lost" from the histogram summation, leading to a CDF that does not perfectly sum to the total tile pixels. While a strictly correct implementation would redistribute this remainder to the first few bins, we deliberately maintained this simplification in the GPU kernel. This ensures that our CUDA output remains **bit-wise consistent** with the provided CPU reference implementation, prioritizing verification correctness.

3 Optimization Evolution (Intermediate Versions)

Instead of jumping directly to the final solution, we adopted an incremental optimization strategy. The commented-out sections in our source code represent distinct stages of this evolution. Below we analyze the bottlenecks identified in each stage and how they drove the design of the final implementation.

3.1 Stage 1: The Baseline Implementation (Naive)

The initial version was a direct translation of the sequential logic to CUDA, prioritizing correctness over performance.

- **Configuration:** It utilized standard Global Memory arrays using `int` (4 bytes) for the LUTs and blocking `cudaMemcpy` transfers. Crucially, it employed a 2D block configuration resulting in 1024 threads per tile.
- **Bottleneck Analysis:** This implementation suffered from severe **Atomic Contention**. With 1024 threads attempting to update a small 256-bin histogram simultaneously, serialization was unavoidable. Additionally, the blocking transfers left the GPU idle for significant periods, exposing the full latency of the PCIe bus.

3.2 Stage 2: Texture Caching (Blocking)

In this iteration, we attempted to address the memory access latency of the rendering kernel by introducing the Texture Unit, while keeping the data transfers synchronous.

- **Configuration:** Replaced Global Memory reads with `tex1Dfetch`, but maintained blocking copies and `int` data types.
- **Observation:** While the texture cache improved the spatial locality of memory accesses during interpolation, the overall application performance did not improve significantly.
- **Root Cause:** The application was **Transfer Bound**. The computation was faster, but the GPU was starving for data because the Host-to-Device transfers were strictly serialized with execution.

3.3 Stage 3: Basic Pipelining (Streams without Textures)

To address the transfer bottleneck identified in Stage 2, we removed the Texture unit and focused on implementing CUDA Streams to overlap transfers with computation.

- **Configuration:** Implemented asynchronous prefetching and execution using 4 streams, but reverted to direct Global Memory access for the LUTs.
- **Observation:** This successfully hid the PCIe transfer overhead, but the kernel execution time itself became the limiting factor.
- **Root Cause:** The rendering kernel hit the **"Memory Wall"**. Without the texture cache, the four non-coalesced memory reads required for bilinear interpolation saturated the global memory bandwidth, preventing the compute units from being fully utilized.

3.4 Stage 4: Data & Thread Optimization (Global Memory)

Before combining all techniques, we refined the data structures and threading model to reduce bandwidth pressure and contention.

- **Configuration:** We demoted the LUT data type from `int` to `unsigned char` (reducing data volume by 75%) and reduced the block size to 256 threads (1D) to minimize atomic collisions in the histogram phase.
- **Observation:** This drastically reduced the memory bandwidth requirements and improved histogram generation speed.
- **Remaining Bottleneck:** Despite the lighter data load, accessing the LUTs from Global Memory (even as bytes) incurs high latency compared to cached reads. The interpolation step remained sensitive to memory delays.

3.5 Stage 5: The Final Version (Combined Strategy)

To eliminate the remaining bottlenecks, we integrated all successful optimizations into a single pipeline.

- **Final Configuration:** We combined the **Pipelining** from Stage 3 (to hide transfer latency), the **Data/Thread Optimizations** from Stage 4 (to reduce bandwidth and contention), and re-introduced **Texture Caching** from Stage 2.
- **Conclusion:** By using Texture Memory to handle random access latency and Streams to handle transfer latency, we ensured that neither the memory subsystem nor the PCIe bus acted as a bottleneck, allowing the GPU to reach its maximum throughput potential.

4 Failed Optimization Attempts (Negative Results)

During the development process, we implemented an intermediate version using a **One-to-One Thread Mapping** strategy that utilized 2D Thread Blocks. This approach was ultimately discarded due to functional correctness issues and poor performance.

4.1 2D Thread Blocks (1024 Threads/Tile)

In this attempt, we mapped exactly one CUDA thread to each pixel of the 32×32 tile, resulting in a block size of `dim3(32, 32)` (1024 threads).

- **Atomic Contention:** With 1024 threads concurrently executing `atomicAdd` on a Shared Memory histogram of only 256 bins, the collision rate was excessively high (ratio 4:1). This caused severe serialization of memory accesses, negating the benefits of parallelism.
- **Wasted Resources & Complex Race Conditions:** The histogram generation and Cumulative Distribution Function (CDF) calculation (Parallel Scan) operate on 256 elements. By using 1024 threads, 75% of the threads (indices 256-1023) were idle during the reduction and mapping phases. Furthermore, managing the synchronization barriers (`__syncthreads()`) between active and idle warps in a 2D block structure created complex race conditions.
- **Correction:** We shifted to a 1D block strategy with **256 threads**. Each thread processes multiple pixels (4 pixels/thread). This drastically reduced atomic contention and simplified the prefix-sum logic, ensuring correctness.

4.2 Direct Global Memory Rendering

Before implementing Texture Memory, the rendering kernel accessed the Look-Up Tables directly via a global integer pointer (`int* d_luts`).

- **Bottleneck:** The bilinear interpolation step requires 4 non-coalesced memory reads per pixel. Without the caching mechanism of Texture Memory, the kernel became strictly memory-bound, exhibiting extremely high latency and low throughput compared to the texture-optimized version.

5 Performance Evaluation

5.1 Experimental Setup

The performance measurements were conducted on an NVIDIA Tesla K80 GPU. We utilized a dataset of 6 PGM images ranging from low resolution (1.5 MP) to high resolution (323 MP) to evaluate the scalability of our implementation. The primary metric for performance is **Throughput**, measured in Mega-Pixels per Second (MPixels/s), calculated as:

$$\text{Throughput} = \frac{\text{Image Width} \times \text{Image Height}}{\text{Processing Time} \times 10^6} \tag{4}$$

All reported timing results are averages 100 execution runs to ensure statistical stability.

5.2 Optimization Impact Analysis

To quantify the effectiveness of each optimization strategy, we analyzed the throughput evolution using the largest dataset (`senator.pgm`, 323.5 MP), which fully saturates the GPU resources. Figure 1 illustrates the performance gains across the five development stages.

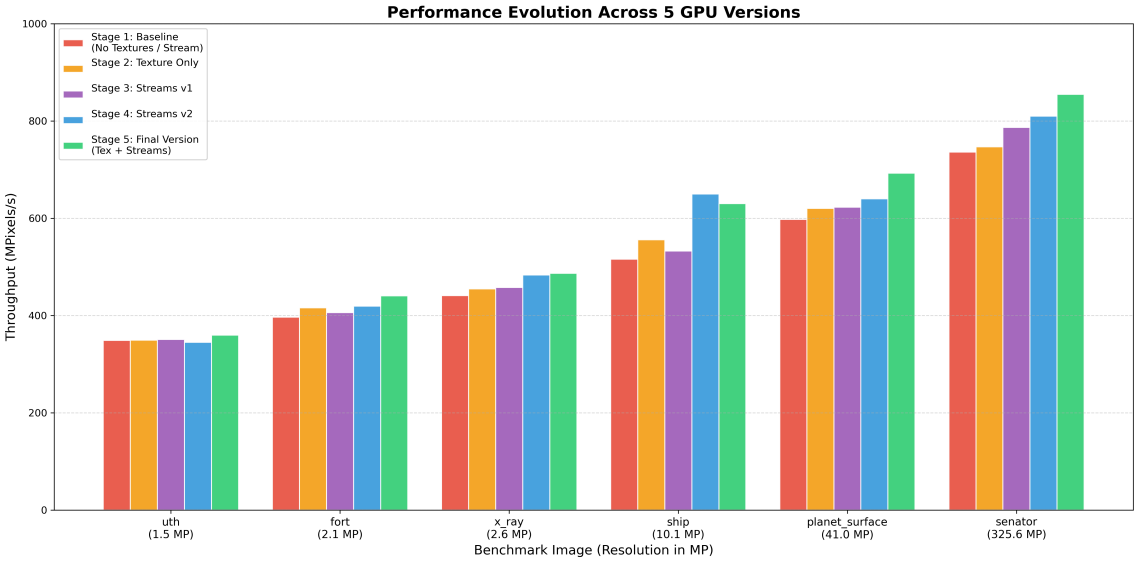

Figure 1: Throughput evolution across the five development stages for the largest dataset (senator.pgm).

Table 2: Throughput evolution on `senator.pgm` (323.5 MP)

Version	Throughput (MP/s)	Improvement vs Baseline
1. Baseline (STAGE 1)	736.03	-
2. Texture Only (STAGE 2)	746.83	+1.5%
3. Streams v1 (STAGE 3)	786.66	+6.9%
4. Streams v2 (STAGE 4)	809.91	+10.0%
5. Final (STAGE 5)	**854.82**	**+16.1%**

Analysis:

- **Texture vs. Streams:** Enabling Texture Memory alone (Stage 2) provided a negligible improvement (+1.5%), whereas enabling Streams alone (Stage 3) yielded a much higher boost (+6.9%). This confirms that the first implementation was primarily **Transfer Bound** (limited by PCIe latency) rather than Compute Bound.
- **Combined Effect:** The Final Version achieves the highest throughput (854.82 MP/s) because it addresses both bottlenecks simultaneously: Streams hide the data transfer latency, while Textures hide the memory access latency during the interpolation phase.

5.3 CPU vs GPU Comparison & Scalability

We compared our final optimized CUDA implementation against the provided reference CPU implementation (compiled with `-O3`). Figure 2 presents the speedup factor achieved across different image sizes.

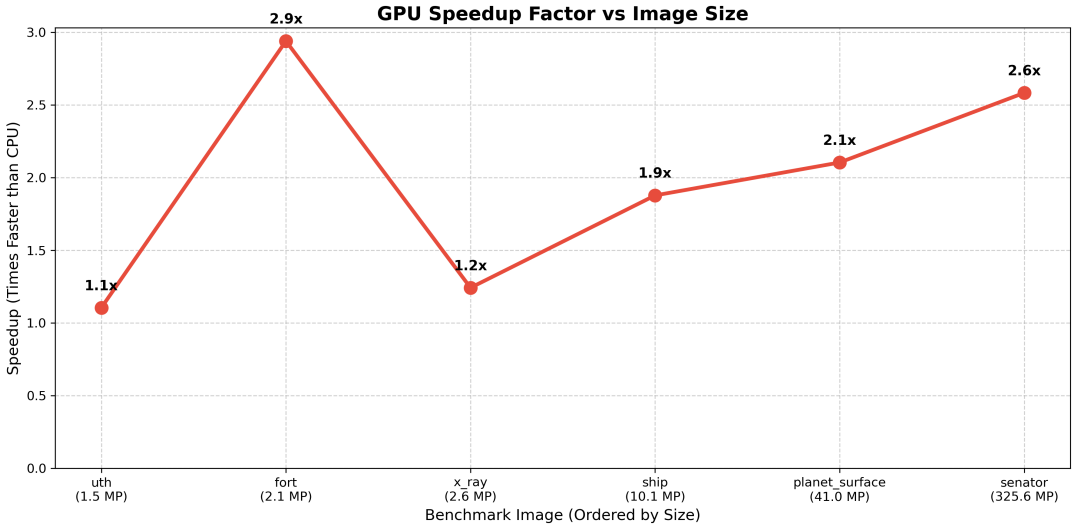

Figure 2: GPU Speedup Factor vs Image Size. The efficiency scales with workload size.

Table 3: CPU vs GPU Performance Comparison

Image	Size (MP)	CPU (MP/s)	GPU Final (MP/s)	Speedup
uth.pgm	1.5	324.94	359.78	1.11x
fort.pgm	2.1	149.54	440.39	2.94x
x_ray.pgm	2.6	390.72	486.55	1.25x
ship.pgm	10.1	330.94	629.96	1.90x
planet.pgm	41.0	328.68	692.43	2.11x
senator.pgm	**323.5**	**328.88**	**854.82**	**2.60x**

Scalability Observation: The results highlight a fundamental difference in how CPU and GPU architectures handle increasing workloads:

1. **CPU Throughput Stability:** As observed in Table 3, the CPU throughput remains remarkably stable (approximately 330 MP/s) regardless of the image size (e.g., `ship.pgm` vs `senator.pgm`). Since the CPU processes pixels sequentially, the execution time scales linearly with the number of pixels, keeping the throughput constant. The CPU hits its performance ceiling early and cannot accelerate further.

2. **GPU Throughput Growth:** In contrast, the GPU throughput *increases* with the workload size.

   - **Small Workloads ($<$ 3 MP):** For small images, the throughput is low ($\approx$ 360 MP/s) because the fixed overhead of CUDA initialization and kernel launching dominates the total runtime. The GPU is effectively underutilized.
   - **Large Workloads ($>$ 40 MP):** As the image size grows, the fixed overhead becomes negligible compared to the computation time. The massive parallelism of the Tesla K80 is fully utilized, allowing the throughput to peak at **854.82** MP/s.

3. **Conclusion:** While the GPU offers little benefit for small images (Speedup $\approx 1.1x$), the performance gap widens drastically for high-resolution data, reaching a Speedup of **2.6x** as the GPU throughput climbs while the CPU remains stagnant.

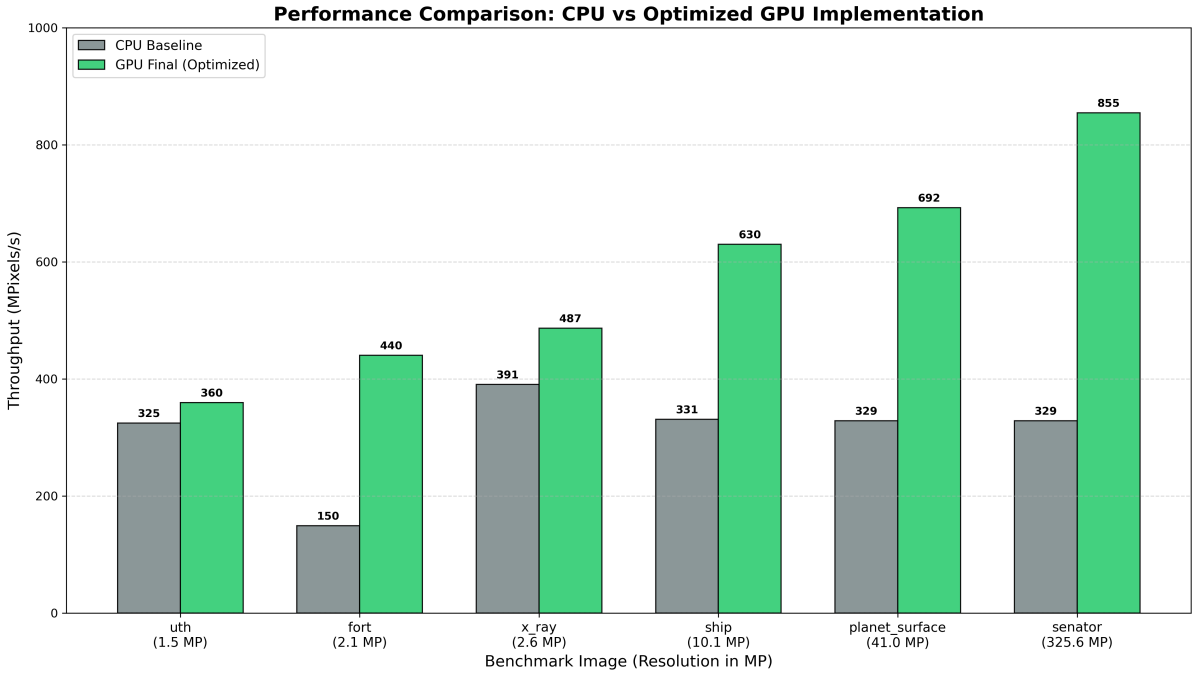

Figure 3: Throughput comparison between CPU Baseline and Final GPU Version across different image resolutions.

6 Conclusion

In this assignment, we successfully designed and optimized a massively parallel implementation of the CLAHE algorithm using CUDA on an NVIDIA Tesla K80 architecture.

Our incremental optimization strategy revealed that the primary bottlenecks were not computational, but related to **Memory Bandwidth** and **Data Transfer Latency**. We successfully addressed these through three key architectural decisions:

1. **Latency Hiding via Pipelining:** By decomposing the workload into streams, we overlapped PCIe data transfers with kernel execution, effectively "hiding" the communication overhead.

2. **Texture Caching:** The use of Texture Memory (`tex1Dfetch`) was the decisive factor in optimizing the rendering kernel. It transformed the global memory reads required for bilinear interpolation into fast, cached accesses.

3. **Data Efficiency:** Demoting the Look-Up Tables from 32-bit integers to 8-bit characters quadrupled the effective memory bandwidth, feeding the CUDA cores at a much higher rate.

Final Verdict: The final implementation achieved a peak throughput of **854.82 MPixels/s**, delivering a speedup of **2.60x** over the optimized CPU reference for high-resolution inputs (325 MP). Our scalability analysis confirms that while GPUs incur a fixed overhead for small tasks, they scale excellently with workload size, making them the superior choice for processing high-definition imaging data.